EventSphere

A full-stack **Community Event Management Platform** enabling organisers to create and manage events, and participants to discover, register, and provide feedback — all within a single centralised hub.

Technology Stack

Backend

- **Node.js + Express.js** — Non-blocking I/O, routing, middleware
- **MongoDB Atlas**— Flexible NoSQL schema with validation
- **JWT** — Stateless authentication
- **bcryptjs** — Password hashing
- **express-validator, Helmet, CORS** — Validation & security headers

Frontend

Built with **HTML, CSS, and Vanilla JavaScript** — no heavyweight frameworks. Static files (`index.html`, `dashboard.html`, etc.) are served directly by Express from the `frontend/` directory.

LIGHTWEIGHT | NO BUILD STEP | FRAMEWORK-FREE

SYSTEM DESIGN

Architecture

Layered Architecture

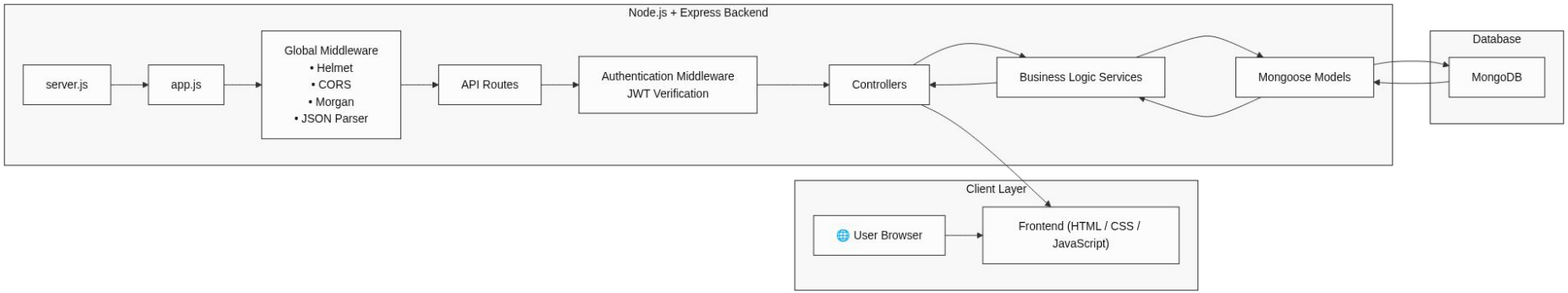
EventSphere follows a strict **Controller–Service–Data Access** pattern, keeping each layer focused and testable.

- **Controllers** — Handle HTTP specifics only (req, res, status codes)
- **Services** — Contain all business rules; controllers stay thin
- **Models** — Manage schema definitions and Mongoose validation

Middleware Pipeline

Routes use composed middleware chains:

```
[protect, restrictTo, validation, validate, controller]
```



Implemented Features

User Authentication

Login, signup, JWT generation, and bcrypt password hashing.

Role-Based Access Control

Distinct permissions for participants, organizers, and admins.

Event Management

Full CRUD for Events and Organizations with validation.

Event Registration

Register, waitlist, cancel, and mark attendance.

Feedback System

One rating and comment per user per event.

Centralized Error Handling

Unified operational error processing across the app.

Security Measures

JWT Authentication

Stateless tokens for secure, scalable session management.

bcrypt Hashing

Passwords never stored in plaintext in the database.

Helmet & CORS

Secures HTTP headers and controls cross-origin access.

express-validator

Strict input validation at route level prevents injection attacks.

DB Compound Indexes

Unique indexes on userId + eventId prevent duplicate registrations and feedback.

Authentication & Authorisation Flow

1

Signup

`POST /api/auth/signup` — Email checked, password hashed via Mongoose pre-save hook, tokens issued.

2

Login

`POST /api/auth/login` — Credentials verified via `bcrypt.compare`; Access + Refresh tokens returned.

3

Protect

`protect` middleware verifies JWT, decodes user ID, fetches user, attaches to `req.user`.

4

RBAC

`restrictTo('organizer', 'admin')` ensures only authorised roles can perform privileged actions.

Error Handling Strategy

This three-layer approach ensures every error — whether a validation failure, a database conflict, or an unexpected exception — is caught, classified, and returned as a consistent, human-readable HTTP response.

API REFERENCE

API Documentation

Authentication

`/api/auth`

- POST /signup — Register
- POST /login — Authenticate
- GET /me — Current user

Events

`/api/events`

- GET / — List events (public)
- GET /:id — Event details
- POST / — Create (organizer)
- PATCH /:id — Update
- DELETE /:id — Delete

Organizations

`/api/organizations`

- GET / — List all
- GET /:id — Details
- POST / — Create (admin)

Registrations

`/api/registrations`

- POST /:eventId/register
- PATCH /:eventId/cancel
- GET /me — My registrations
- GET /:eventId — All (organizer)
- PATCH /:eventId/attendance/:userId

Feedback

`/api/feedback`

- POST /:eventId — Submit (one per user)
- GET /:eventId — View (public)

REFERENCE

Repository Inventory

Controllers & Routes

- **auth** — authController.js / authRoutes.js
- **events** — eventController.js / eventRoutes.js
- **feedback** — feedbackController.js / feedbackRoutes.js
- **orgs** — orgController.js / orgRoutes.js
- **registrations** — registrationController.js / registrationRoutes.js

Middleware

- **auth.js** — protect (JWT)
- **rbac.js** — restrictTo (roles)
- **validate.js** — validation wrapper
- **errorHandler.js** — catchAsync, AppError, global handler

Models

- Event.js · Feedback.js · Organization.js
- Registration.js · User.js

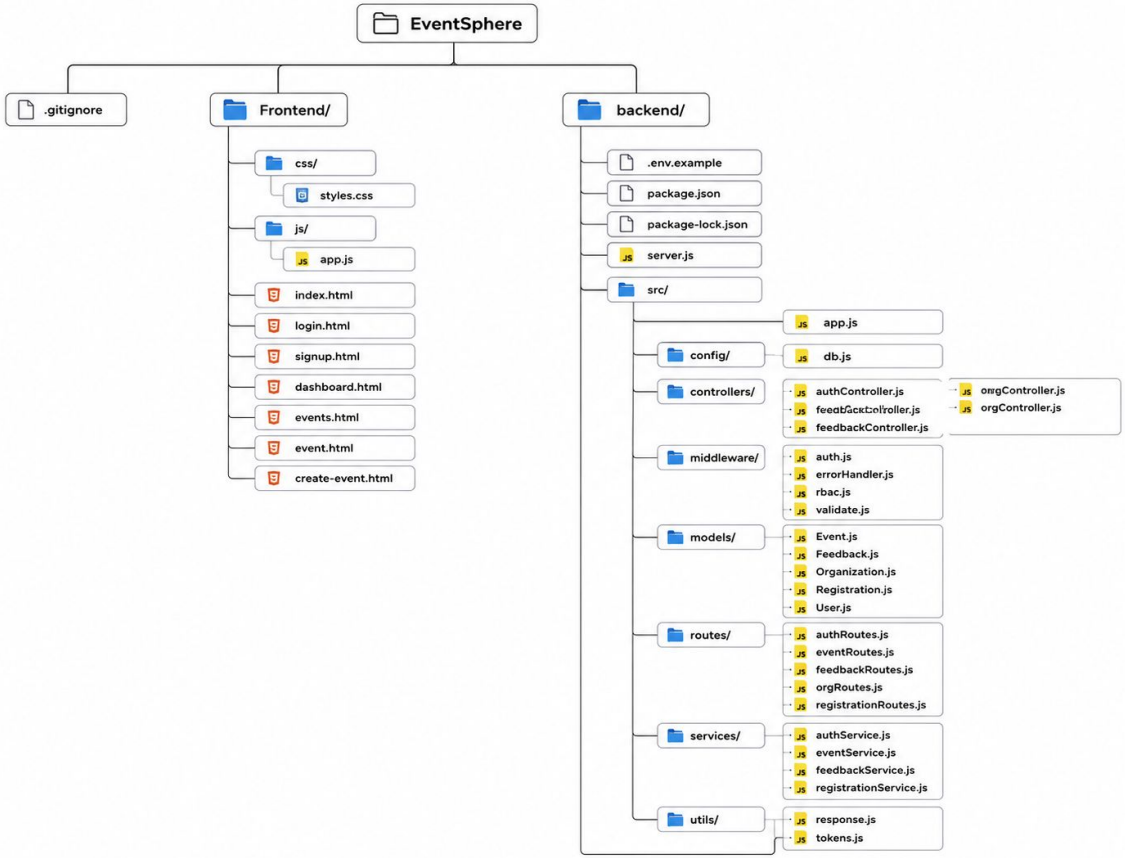
Services

- authService.js · eventService.js
- feedbackService.js · registrationService.js

Utilities & Config

- **response.js** — Standard JSON responses
- **tokens.js** — JWT generation
- **db.js** — MongoDB connection
- **server.js** · **app.js** · **.env.example** · **package.json**

Repository Folder Structure



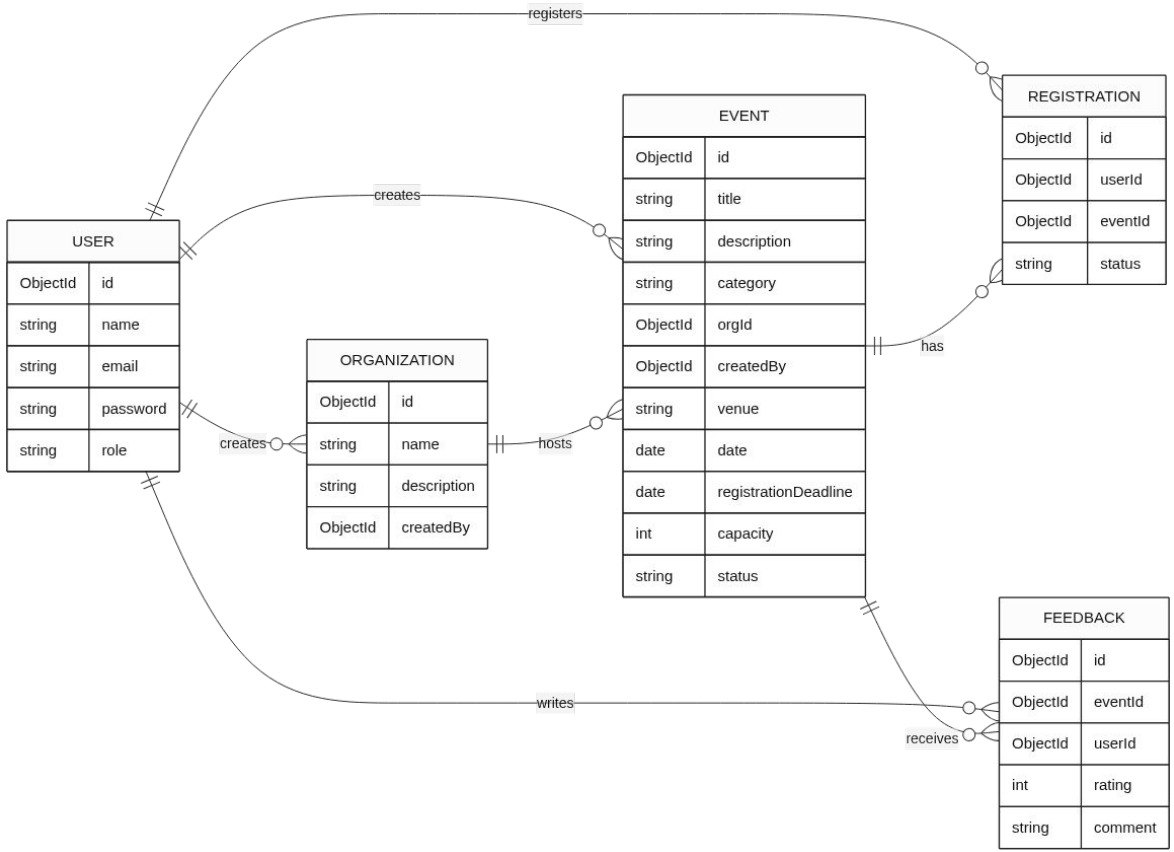
Backend – Core Logic

The `backend/` folder follows a clean MVC-like separation. `server.js` bootstraps the app; `src/` houses all source code split across controllers, services, routes, models, middleware, and utils.

Frontend – Static Assets

The `frontend/` folder contains HTML pages and their associated `css/` and `js/` subdirectories, served statically by Express.

Database Design



Key Schema Highlights

- **User** — Role field drives RBAC; password excluded from query results by default.
- **Event** — Compound index on `{status, date}` for fast published-event queries.
- **Registration** — Unique `{userId, eventId}` index prevents duplicate registrations.
- **Feedback** — Unique `{userId, eventId}` index enforces one review per user per event.

Request Lifecycle

Client Request | Global Middleware | Route Middleware | Controller → Service

Every request flows through a well-defined pipeline. Global middleware applies first, then route-level middleware handles authentication, authorisation, and validation. The Controller delegates business logic to the Service layer, which interacts with MongoDB via Mongoose. Errors at any stage are caught by `catchAsync` and handled centrally.

Future Improvements

1 Refresh Token Endpoint

Consume refresh tokens to issue new access tokens without re-authentication.

2 Security Hardening

Add `express-rate-limit` on auth routes and `xss-clean` for data sanitization.

3 Pagination & Filtering

Support `limit`, `page`, and `sort` query params on `GET /api/events`.

4 Automated Tests

Introduce Jest or Mocha/Chai — the repository currently has no test coverage.

Conclusion

EventSphere delivers a robust, secure, and scalable platform for seamless event management.

Full-Featured Platform

Comprehensive API covering authentication, event lifecycle, and user interactions.

Secure & Reliable

Robust authentication, role-based access, and input validation ensure data integrity.

Modular & Scalable

Layered architecture designed for maintainability and future expansion.

Unified Error Handling

Consistent error responses simplify integration and debugging for developers.

This foundation provides a powerful and extensible solution for your event ecosystem needs.